

Sankhya — the complete account

From the mathematics up: what is being built, how it works, what was rejected and why

Mohit Prajapati

30 August 2026

Contents

How to read this	4
The symbols, once	4
Part I — The problem	5
1. What MRPL actually asked for	5
2. The tiny refinery we will use throughout	5
3. Three problem classes, and why all three are needed	6
Part II — The mathematics, from scratch	7
4. Theorem 1 — the answer is always at a corner	7
4.1 In plain words	7
4.2 The statement	7
4.3 Why this matters more than it looks	7
5. Theorem 2 — duality	7
5.1 The guessing game	8
5.2 What just happened, in general	8
5.3 Weak duality	9
5.4 Strong duality	9
5.5 Shadow prices — what the dual numbers <i>are</i>	10
5.6 Reduced costs — “is this product worth making?”	10
5.7 Complementary slackness	11
6. Theorem 3 — Farkas’ lemma, or how to prove there is <i>no</i> answer	11
6.1 On numbers first	11
6.2 The statement	12
6.3 What the solver does with it	12
7. Why whole numbers make it hard	12
7.1 The obvious idea, and why it fails	12
7.2 What is done instead: branch and bound	13
7.3 And why a wrong bound is a catastrophe	13
8. Projection — the one operation the GPU method needs	14
Part III — The methods, and why these ones	15
9. Three ways to solve an LP	15
10. The simplex method, concretely	15

10.0	In plain words first	15
10.1	The ratio test, written out	16
10.2	Devex, written out	17
10.3	The two refinements that matter most here	18
10.4	The dual simplex — the one that actually runs the MILP	18
10.5	Underneath everything: the LU factorisation	19
11.	The first-order method — the piece that runs on a GPU	20
11.1	Why a saddle point	20
11.2	PDHG	20
11.3	Restarts and Halpern	21
11.4	What this method actually is	22
11.5	Feasibility polishing	22
11.6	Scaling — the step that decides whether any of it works	22
11.7	When to stop — the termination test	23
11.8	Proving there is no answer	24
11.9	Step size, primal weight, restarts — the three tuned quantities	24
12.	Branch and cut, concretely	26
12.1	Cuts	26
12.2	Reduced-cost fixing	27
12.3	Node propagation	27
12.4	Reliability branching	28
13.	QP by ADMM, written out	28
13.1	The splitting	28
13.2	The linear system, and why it always factorises	29
13.3	The factorisation itself	29
13.4	Adaptive ρ , and where it breaks	30
13.5	Polishing	30
14.	Presolve, with the algebra	30
14.1	Postsolve is a replay, not an inverse	32
15.	How the pieces fit together	32
Part IV — Every option that was rejected, and why		33
16.	Rejected, with the numbers	33
16.1	Interior point methods — the full argument	33
16.2	Hypersparsity	34
16.3	Forrest–Tomlin updates	34
16.4	Bound-flipping ratio test	34
16.5	Coefficient tightening	34
16.6	Parallel column merging	34
16.7	Harris ratio test without EXPAND	34
16.8	Two fixes for a QP convergence failure	35
Part V — What went wrong, and what it taught		36
17.	Six times the solver was confidently wrong	36
18.	The tools that exist because something got through	36
19.	Traps in measuring, not in the code	37
Part VI — Where it stands and what is next		38
20.	Verified against published answers	38

21. The benchmark to be judged against	38
22. What is next, in order	38
23. The one thing worth taking away	38

How to read this

Nothing here assumes you have seen optimisation before. If you can solve two equations in two unknowns and you are comfortable with an inequality like $2x + 3y \leq 12$, you have enough.

Every theorem is done twice: once in plain words with actual numbers you can check on paper, and once in the general form the code uses. If a formula looks heavy, the sentence above it says the same thing in English — read that first and come back.

One example runs through the whole document. A tiny refinery with two products. It is small enough to solve by hand, and every idea — duality, shadow prices, reduced costs, branching, cuts — will be demonstrated on it before being stated in general.

The symbols, once

Optimisation is written with vectors and matrices because a refinery has ten thousand variables and nobody is writing those out one at a time. But every symbol below is shorthand for something you already know.

Symbol	Means
x	a vector — just a list of numbers, $x = (x_1, x_2, \dots, x_n)$. Our decisions.
c	another list, the same length — the profit or cost of each decision
A	a matrix — a table of numbers, one row per rule, one column per decision
A_{ij}	the number in row i , column j of that table
$c^\top x$	multiply matching entries and add: $c_1x_1 + c_2x_2 + \dots$
Ax	do that for every row at once: row i gives $A_{i1}x_1 + A_{i2}x_2 + \dots$
$A^\top$	the table flipped — rows become columns
$x \geq 0$	every entry of x is ≥ 0
$\ x\ $	the length of the list: $\sqrt{x_1^2 + x_2^2 + \dots}$ — Pythagoras, extended
$\sum_j$	“add up over all j ”
$\min_x f(x)$	the smallest value f can take, choosing x
$\Pi_{[l,u]}(v)$	clamp v into $[l, u]$: below l becomes l , above u becomes u

So when you see

$$\min_x c^\top x \quad \text{such that} \quad Ax \leq b, \quad x \geq 0$$

read it as: *choose the numbers x , all of them non-negative, so that every rule in the table A is respected, and the total cost is as small as possible.* That is the entire subject.

Part I — The problem

1. What MRPL actually asked for

Problem statement SIH26119: an **indigenous, GPU-accelerated optimization solver** for refinery planning.

Unpack that into what it means to build.

A refinery buys several crude oils, runs them through units (distillation, reformer, cracker), and blends the intermediate streams into products — petrol, diesel, jet fuel. Every decision is a number: how many barrels of Bombay High to run, how much naphtha goes to the reformer instead of into the gasoline pool, how much of each stream ends up in each product.

Those numbers are not free. Each unit has a capacity. Every stream that enters a unit must leave it (material balance). Every product has specifications it must meet — octane, sulphur, density. And you want the combination that makes the most margin.

Written down, that is exactly a **linear program**, and the refinery industry has known this since the 1950s. What changes is scale and speed: a real planning model has tens of thousands of constraints, and planners want to re-run it many times a day with different crude prices.

“Indigenous” means: do not wrap Gurobi or CPLEX. Build the solver.

“GPU-accelerated” means: it has to actually use the hardware, which turns out to constrain the choice of algorithm heavily — more on that in Part III.

2. The tiny refinery we will use throughout

Strip a refinery down to the smallest thing that still behaves like one.

We make two products, in kilolitres per day:

- x_1 = petrol
- x_2 = diesel

Petrol earns Rs 4 per kL of margin, diesel Rs 3. We want the most margin.

Three limits:

	rule	why
Unit A (cracker)	$2x_1 + x_2 \leq 100$	petrol needs 2 hours per kL here, diesel 1; 100 hours a day
Unit B (blender)	$x_1 + x_2 \leq 80$	both need 1 hour per kL; 80 hours a day
Market	$x_1 \leq 40$	we can only sell 40 kL of petrol

And you cannot make a negative amount: $x_1, x_2 \geq 0$.

The whole problem:

$$\max 4x_1 + 3x_2 \quad \text{s.t.} \quad \begin{cases} 2x_1 + x_2 \leq 100 \\ x_1 + x_2 \leq 80 \\ x_1 \leq 40 \\ x_1, x_2 \geq 0 \end{cases}$$

Solve it by hand. Try the corners of the allowed region:

plan	Unit A used	Unit B used	margin
(0, 0)	0	0	0
(40, 0)	80	40	160
(40, 20)	100	60	220
(0, 80)	80	80	240
(20, 60)	100	80	260

The best plan is **20 kL petrol and 60 kL diesel, margin Rs 260/day**, and it uses Unit A and Unit B completely while leaving the market limit slack (we sell 20 of the 40 petrol we could).

Keep those numbers. Everything in Part II is proved on them.

3. Three problem classes, and why all three are needed

Linear program (LP). Everything linear, like the example above.

$$\min_x c^\top x \quad \text{subject to} \quad l \leq Ax \leq u, \quad lo \leq x \leq hi$$

(Writing it as a minimisation is the convention; maximising $4x_1 + 3x_2$ is the same as minimising $-4x_1 - 3x_2$. The solver works in min form and flips the sign back at the end.)

Mixed-integer program (MILP). The same, but some variables must be whole numbers:

$$x_j \in \mathbb{Z} \quad \text{for } j \in I$$

Needed the moment a decision is *yes or no* rather than *how much*: run this unit or not, buy this crude cargo or not, use this blending recipe or not. Also for anything piecewise-linear, which is how a nonlinear blending curve gets approximated.

Quadratic program (QP). A squared term in the objective:

$$\min_x \frac{1}{2}x^\top Qx + c^\top x$$

A squared term is what you write when you are penalising *deviation*. “Stay close to last month’s plan” means minimise $(x - x_{\text{last}})^2$, and a squared error is quadratic. Risk is quadratic too.

A planning tool that only does LP is a demo. A refinery needs at least LP and MILP, and QP the moment anything is a least-squares fit.

Part II — The mathematics, from scratch

Five ideas. Corner solutions, duality, Farkas' lemma, why integers are hard, and projection. Every method in Part III is one of these five turned into a loop.

4. Theorem 1 — the answer is always at a corner

4.1 In plain words

Draw the tiny refinery on graph paper. x_1 across, x_2 up. Each rule $2x_1 + x_2 \leq 100$ is a straight line with everything on one side of it allowed. Three rules plus the two non-negativity rules cut out a five-sided region — the set of plans you are permitted to run.

Now the margin. $4x_1 + 3x_2 = 120$ is also a straight line. So is $4x_1 + 3x_2 = 200$, and $= 260$. They are all **parallel** — same slope, different positions. Higher margin means further out.

So the question becomes: slide a ruler outward, keeping it parallel, until it is just about to leave the region. Where does it last touch?

It touches at a corner. It has to. If it touched only at a point in the middle of an edge, you could slide it a hair further and still touch — so that was not the last touch. The only way to get stuck is at a corner, or (if the ruler happens to be exactly parallel to an edge) along a whole edge — and then the two corners at its ends are just as good.

4.2 The statement

Theorem. If a linear program has an optimal solution, then at least one optimal solution is a **vertex** (corner) of the feasible region.

4.3 Why this matters more than it looks

A region has infinitely many points and only finitely many corners. This theorem turns “search an infinite set” into “check a finite list”.

In our example the corners were $(0, 0)$, $(40, 0)$, $(40, 20)$, $(0, 80)$, $(20, 60)$ — five of them, and we found the answer by checking all five.

But the count explodes. A corner is where n of the rules hold with equality at once, so with m rules and n variables there can be up to $\binom{m}{n}$ corners. For a small refinery model with 200 rules and 100 variables that is more corners than there are atoms in the observable universe. Checking all of them is not a plan.

The **simplex method** (Section 10) is the fix: start at one corner and walk to a neighbouring corner that is better, repeatedly. It never enumerates; it climbs. And because the region has no dents — it is *convex*, being an intersection of half-planes — a corner with no better neighbour is the global best, not a local trap. That last sentence is the whole reason the walk is allowed to stop.

5. Theorem 2 — duality

This is the most important idea in the document. The simplex's stopping rule, branch-and-bound's pruning, shadow prices, and half of presolve are all this one idea wearing different clothes. So it is worth doing slowly, on the numbers we already have.

5.1 The guessing game

You claim the best possible margin is Rs 260 per day. I do not believe you. **How do you prove no plan can beat 260, without checking every plan?**

Here is the trick, and it is genuinely all there is to duality.

Our rules are:

$$(A) \quad 2x_1 + x_2 \leq 100 \quad (B) \quad x_1 + x_2 \leq 80 \quad (C) \quad x_1 \leq 40$$

Multiply any of them by a **non-negative** number and they stay true. Add true inequalities and they stay true. So let us mix them and see what falls out.

First attempt. Take rule (B) and multiply by 4:

$$4x_1 + 4x_2 \leq 320$$

Now, since $x_2 \geq 0$ we know $3x_2 \leq 4x_2$, and therefore

$$4x_1 + 3x_2 \leq 4x_1 + 4x_2 \leq 320$$

Margin can never exceed 320. That is a proof. It took one line and no searching. It is also loose — 320 is well above the 260 we found.

Second attempt. Take $1 \times$ rule (A) plus $2 \times$ rule (B):

$$\begin{array}{r} 1 \times (2x_1 + x_2) \leq 1 \times 100 \\ 2 \times (x_1 + x_2) \leq 2 \times 80 \\ \hline 4x_1 + 3x_2 \leq 260 \end{array}$$

Look at the left side: $2x_1 + x_2 + 2x_1 + 2x_2 = 4x_1 + 3x_2$. That is **exactly the margin**, no slack, nothing thrown away. So:

$$\text{margin} \leq 260$$

And we already have a plan that *achieves* 260. Therefore 260 is optimal, and we have proved it. Nobody has to trust the search.

5.2 What just happened, in general

We chose a weight for each rule — call them y_1, y_2, y_3 , all ≥ 0 — and added up the weighted rules. For the result to be a bound on $4x_1 + 3x_2$, the weighted left side has to dominate the objective coefficient by coefficient:

$$\begin{aligned} \text{coefficient of } x_1 : \quad & 2y_1 + y_2 + y_3 \geq 4 \\ \text{coefficient of } x_2 : \quad & y_1 + y_2 \geq 3 \end{aligned}$$

and then the bound we get is the weighted right side, $100y_1 + 80y_2 + 40y_3$.

Naturally we want the *tightest* such bound — the smallest one. That is itself an optimisation problem:

$$\min 100y_1 + 80y_2 + 40y_3 \quad \text{s.t.} \quad \begin{cases} 2y_1 + y_2 + y_3 \geq 4 \\ y_1 + y_2 \geq 3 \\ y_1, y_2, y_3 \geq 0 \end{cases}$$

This is the dual problem. The original is the **primal**. Notice what happened structurally, because it is worth recognising:

- one dual variable per primal **rule** (3 rules $\rightarrow$ 3 dual variables)
- one dual rule per primal **variable** (2 variables $\rightarrow$ 2 dual rules)
- the table A appears flipped: $A^\top$
- max becomes min, $\leq$ becomes $\geq$
- the objective and the right-hand side swap jobs

Our second attempt was $y = (1, 2, 0)$. Check it: $2(1) + 2 + 0 = 4 \geq 4$ ✓ and $1 + 2 = 3 \geq 3$ ✓, giving $100(1) + 80(2) + 40(0) = 260$. Our first attempt was $y = (0, 4, 0)$, giving 320. Both legal, one better.

5.3 Weak duality

Theorem (weak duality). Every feasible y gives a bound on every feasible x . For a maximising primal: $c^\top x \leq b^\top y$.

Proof, three lines, and it is exactly what we did above:

$$c^\top x \leq (A^\top y)^\top x = y^\top (Ax) \leq y^\top b$$

The first $\leq$ holds because $A^\top y \geq c$ and $x \geq 0$ — replacing each coefficient by something bigger, multiplied by something non-negative, can only increase. The middle is just rebracketing. The last $\leq$ holds because $Ax \leq b$ and $y \geq 0$.

No assumptions, no conditions. Any dual-feasible y is a certificate anyone can check with one multiplication.

Consequence worth naming: the primal is always below the dual. There is a “gap” between them, and it never goes negative.

5.4 Strong duality

Theorem (strong duality). If both problems have feasible points, then at the optimum the two are **equal**: $c^\top x^* = b^\top y^*$.

The gap closes completely. The best proof is exactly as strong as the truth.

This is not obvious and it is not free — the proof needs the separating hyperplane theorem, or Farkas’ lemma from Section 6. But its *consequence* is the thing to remember:

A solver can prove it is done. It produces a plan x and a set of weights y , checks that both are feasible, checks that their objectives match, and hands you all three. If the numbers match, no better plan exists. That is what “optimal” means in this document — not “the search stopped”, but “here is the proof, check it yourself”.

Every claim of optimality in this project is that check.

5.5 Shadow prices — what the dual numbers *are*

We found $y = (1, 2, 0)$. Those numbers mean something concrete.

$y_1 = 1$ belongs to Unit A, which has 100 hours. Suppose we rented one more hour, making it 101. Re-solve: the new optimum is 261. **One more hour of Unit A is worth Rs 1.**

$y_2 = 2$ belongs to Unit B. One more hour there and the margin goes to 262. **Unit B hours are worth twice as much as Unit A hours.**

$y_3 = 0$ belongs to the market limit of 40 kL petrol. We are only selling 20, so raising the limit to 41 changes nothing. **It is worth zero.**

That is why y_i is called the **shadow price** of a constraint: the margin per extra unit of that resource. It answers “where should we spend capital?” and it falls out of the same solve, for free.

If a plant manager can only debottleneck one unit, this tells them which — and notice it is Unit B, which is *not* the one with fewer hours. Intuition would have got that wrong; the dual does not.

This is why the solver works hard to recover duals through presolve, and says so when a recovery is only approximate. Someone may size capital on these numbers. A shadow price you cannot trust is worse than none.

5.6 Reduced costs — “is this product worth making?”

Suppose someone proposes a third product, kerosene (x_3), margin Rs 2 per kL, needing 1 hour on Unit A and 1 hour on Unit B.

Should we make it? Do not re-solve. Just price the ingredients using the shadow prices we already have:

$$\text{what it consumes} = 1 \times \underbrace{y_1}_1 + 1 \times \underbrace{y_2}_2 = 3$$

Kerosene *earns* 2 but *uses up* 3 rupees’ worth of capacity. Making it loses Rs 1 per kL. Do not make it.

That number — earnings minus internal cost — is the **reduced cost**:

$$d_j = c_j - (A^\top y)_j$$

(Our sign convention is minimisation, so in the code $d_j > 0$ means “not worth using”. In the maximising story above the sign is flipped; the meaning is identical.)

The reduced cost is the single most-used quantity in the whole solver:

- **The simplex** prices on it. A variable with a favourable reduced cost is worth bringing into the plan; when none is left, the plan is optimal.
- **Presolve** fixes variables using it.
- **Branch-and-bound** eliminates variables with it (Section 12.2).

5.7 Complementary slackness

Put Sections 5.5 and 5.6 together and a pattern appears:

- Unit A is **fully used** (100 of 100 hours) and its price is **non-zero** (1)
- Unit B is **fully used** (80 of 80 hours) and its price is **non-zero** (2)
- The market limit is **not** fully used (20 of 40) and its price is **zero**

Theorem (complementary slackness). At the optimum, for every constraint: either it is tight, or its dual value is zero. And for every variable: either it is strictly between its bounds, or its reduced cost may be non-zero.

$$y_i \cdot (\text{slack in row } i) = 0 \quad \text{and} \quad d_j \cdot (\text{distance of } x_j \text{ from its bound}) = 0$$

In plain words: *a resource you are not using up is free, and anything you are actively doing must be exactly break-even at the margin.* Both halves are obvious once said aloud — if the market cap were worth something we would be pushing against it; if kerosene were worth making we would already be making some.

This is the **optimality test the solver runs on itself**. It is not decoration. The dual simplex once returned `fit1p` as 33,609 when the true answer is 9,146.38 — feasible, row violation 2.8×10^{-14} , and completely wrong. The check that caught it is this one, and it caught it on the first run. Section 17 tells that story.

6. Theorem 3 — Farkas’ lemma, or how to prove there is *no* answer

A solver that can only say “here is the best plan” is half a tool. Planners over-constrain models constantly. The useful reply is not “I could not find anything” — that might just mean the solver gave up. It is *“these rules contradict each other, and here is the proof.”*

6.1 On numbers first

Suppose someone hands you these requirements:

$$2x_1 + x_2 \leq 10, \quad x_1 + 2x_2 \leq 10, \quad x_1 + x_2 \geq 8, \quad x_1, x_2 \geq 0$$

Is there a plan? You could search for a while and find nothing, which proves nothing. Instead, add the first two rules together:

$$3x_1 + 3x_2 \leq 20 \quad \Rightarrow \quad x_1 + x_2 \leq 6.67$$

But the third rule demands $x_1 + x_2 \geq 8$. And $8 > 6.67$. **Contradiction.**

Notice the shape of that proof: it is a list of multipliers — $\frac{1}{3}$ on the first rule, $\frac{1}{3}$ on the second, 1 on the third — and anyone can verify it in one line without trusting me at all. That list is the whole proof object.

6.2 The statement

Farkas’ lemma. Exactly one of these is true:

$$(i) \exists x \geq 0 \text{ with } Ax = b \quad \text{or} \quad (ii) \exists y \text{ with } A^\top y \leq 0 \text{ and } b^\top y > 0$$

“Exactly one” is the strong part. Not just *at most* one — they cannot both hold, which is easy — but *at least* one. **If no solution exists, a proof of that always exists**, and it is always this simple form.

Why they cannot both hold: if $Ax = b$ with $x \geq 0$, then $b^\top y = (Ax)^\top y = x^\top (A^\top y) \leq 0$, since $x \geq 0$ and $A^\top y \leq 0$. That contradicts $b^\top y > 0$.

6.3 What the solver does with it

The y in case (ii) is an **infeasibility certificate**. The solver attaches it to the answer, and it can be checked with a single matrix-vector product by someone who does not trust the solver at all.

It is also where strong duality (Section 5.4) actually comes from — apply Farkas to the system “primal feasible AND dual feasible AND objectives equal”, and the alternative turns out to be impossible.

And it is the practical difference between a solver saying “*infeasible*” and a solver saying “*rows 4, 17 and 88 cannot all hold — here are the multipliers.*” Only the second is useful to the person who has to fix the model.

7. Why whole numbers make it hard

7.1 The obvious idea, and why it fails

Take this problem, all integers:

$$\max 5x_1 + 4x_2 \quad \text{s.t.} \quad 4x_1 + 5x_2 \leq 20, \quad 3x_1 + 2x_2 \leq 12, \quad x_1, x_2 \geq 0 \text{ integer}$$

Step 1: ignore the integer requirement and solve the LP. Both rules bind, and solving the two equations gives

$$x_1 = 2.857, \quad x_2 = 1.714, \quad \text{margin} = 21.14$$

Step 2: round it. This is what everybody tries first.

rounded plan	rule 1 (≤ 20)	rule 2 (≤ 12)	margin
(3, 2)	22 $\times$	13 $\times$	infeasible
(3, 1)	17 $\checkmark$	11 $\checkmark$	19
(2, 2)	18 $\checkmark$	10 $\checkmark$	18
(2, 1)	13 $\checkmark$	8 $\checkmark$	14

Best from rounding: **19**.

Step 3: the actual answer. It is (4, 0), with margin **20**.

Check it: $4(4) + 5(0) = 16 \leq 20 \checkmark$, $3(4) + 2(0) = 12 \leq 12 \checkmark$.

The true answer has $x_2 = 0$, while the LP said $x_2 = 1.714$. Rounding was never going to find it — the integer optimum is not near the fractional one. The geometry from Section 4 has broken: the feasible set is no longer a region with corners, it is a scatter of dots inside one, and the best dot need not be near any corner.

There is no known algorithm that solves this in polynomial time. MILP is **NP-hard**.

7.2 What is done instead: branch and bound

1. **Relax.** Drop the integer requirement and solve the LP. Here: 21.14. This is a genuine **bound** — the LP is allowed strictly more plans than the MILP, so the true integer answer can never exceed it.
2. **Branch.** x_1 came out 2.857, which no integer can be. Every integer solution has either $x_1 \leq 2$ or $x_1 \geq 3$. Two subproblems. Between them they cover every possibility, and neither contains 2.857.
3. **Bound and prune.** Solve each subproblem's LP. If a subproblem's bound is already no better than the best integer plan found so far, **nothing inside it can beat that plan** — throw the whole subproblem away without exploring it.
4. Repeat on whatever is left.

Run it: the $x_1 \geq 3$ branch gives an LP bound of 20.67, the $x_1 \leq 2$ branch gives 18. Once we have found (4, 0) with 20, the $x_1 \leq 2$ branch is dead immediately — its *best possible* is 18, worse than 20 in hand. An entire half of the tree disappears on one comparison.

The whole game is step 3. A tighter bound prunes more. That is why cuts, presolve and good branching matter so much: every one of them is a way of pushing the bound closer to the truth so that more of the tree evaporates.

7.3 And why a wrong bound is a catastrophe

Step 3 discards subtrees *without looking inside them*. That is its power and its danger. If a bound is wrong by even a rounding error in the unsafe direction, the solver throws away the subtree containing the answer — and then confidently proves optimality for something that is not optimal, with a matching dual bound and no complaint at all.

This happened. On **fiber** the solver returned 652,748.78 against a true 405,935.18 — **60.8% wrong**, and it looked perfectly healthy. Section 17 has the two bugs.

8. Projection — the one operation the GPU method needs

The last idea, and the shortest.

Projection onto a set means: given a point, find the nearest point that is allowed. For an interval it is just clamping.

$$\Pi_{[0,10]}(13) = 10, \quad \Pi_{[0,10]}(-4) = 0, \quad \Pi_{[0,10]}(7) = 7$$

That is it. Three cases, no loops, no dependence on any other entry of the vector. And because there is no dependence, projecting a million numbers is a million independent operations — which is exactly the shape of work a GPU is built for.

The method in Section 11 is built entirely out of two operations: *multiply by the matrix* and *clamp*. Both are perfectly parallel. That is not a coincidence, it is the design constraint that chose the algorithm.

The useful property, and the reason the method converges at all:

Projection onto a convex set never increases distance. If p and q are two points,
 $\|\Pi(p) - \Pi(q)\| \leq \|p - q\|$.

Clamping two numbers can only bring them closer together, never push them apart — try it. So each iteration is a step that can overshoot, followed by a clamp that cannot make things worse. Repeat enough times and you converge. Section 11.5 uses exactly this property to justify feasibility polishing.

Part III — The methods, and why these ones

Part II gave five ideas. Each method below is one of them turned into a loop that a computer can run for a million iterations.

9. Three ways to solve an LP

	how it works	strength	weakness
Simplex	walk vertex to vertex	exact; warm starts	sequential, hard to parallelise
Interior point	cut through the middle	fast on large problems	needs a sparse factorisation each step; no warm start
First-order	matrix-vector products only	massively parallel; GPU	slow to high accuracy

This project implements **simplex** and **first-order**. Section 16.1 gives the full argument for why interior point was left out, from both sides.

10. The simplex method, concretely

10.0 In plain words first

Section 4 said the answer is at a corner and the trick is to walk from corner to corner. The simplex method is that walk. Everything below is bookkeeping for it.

What is a corner, in numbers? In our tiny refinery, the best plan (20, 60) sat where Unit A and Unit B were *both* exactly full. A corner is where enough rules hold with equality to pin the point down completely — two rules for two variables, n rules for n variables.

The method’s bookkeeping turns that around. Rewrite every $\leq$ rule as an equality by adding a **slack** variable — leftover capacity:

$$2x_1 + x_2 + s_A = 100, \quad x_1 + x_2 + s_B = 80, \quad x_1 + s_M = 40$$

Now there are five variables (x_1, x_2, s_A, s_B, s_M) and three equations. Pick **three** variables to be “in the plan” (**basic**) and force the other two to zero (**nonbasic**). Three equations, three unknowns — solve, and you get a corner. That is the entire correspondence:

a choice of which variables are allowed to be non-zero = a corner

At our optimum, $s_A = 0$ and $s_B = 0$ (both units full), so the basic set is $\{x_1, x_2, s_M\}$ and the nonbasic set is $\{s_A, s_B\}$. Solving gives $x_1 = 20$, $x_2 = 60$, $s_M = 20$ — the 20 kL of unsold petrol capacity.

Walking to the next corner then means: let one nonbasic variable start rising from zero (it *enters*), and stop when some basic variable is forced down to a bound (it *leaves*). Swap them. That is one iteration, and it is one step along one edge of the region.

Three questions per step, and the rest of Section 10 is the three answers:

1. Which variable should enter? — **pricing** (Section 10.2)
2. How far can it go before something breaks? — **the ratio test** (Section 10.1)
3. How do we re-solve the three equations cheaply after the swap? — **the factorisation** (Section 10.5)

Now the same thing in the notation the code uses.

Write the LP with slacks so that $Ax = b$, $x \geq 0$, with A having m rows.

Choose m columns to form an invertible **basis** B ; the rest are nonbasic and sit at a bound. Then

$$x_B = B^{-1}b$$

and the nonbasic variables are determined by which bound they are at.

One iteration:

1. **Price.** Compute duals $y^\top = c_B^\top B^{-1}$ and reduced costs $d_j = c_j - y^\top A_j$ for nonbasic j . If every d_j has the right sign (Section 5.7), we are optimal — stop.
2. **Choose an entering variable** q with a favourable d_q .
3. **Ratio test.** Increasing x_q changes the basics along $\alpha_q = B^{-1}A_q$. Increase x_q until the first basic variable hits a bound. That variable leaves.
4. **Pivot.** Swap them, update B^{-1} .

Three things dominate the cost, and each has a name in the code:

- **FTRAN** — solving $B\alpha = A_q$
- **BTRAN** — solving $B^\top y = c_B$
- **PRICE** — forming d_j for every nonbasic column

10.1 The ratio test, written out

In plain words. We are raising one variable from zero. Everything else moves in response — some go up, some go down. The question is: *which one hits a wall first, and how far did we get?*

On our refinery, suppose we are at $(0,0)$ and start raising diesel x_2 . Unit A allows $x_2 \leq 100$, Unit B allows $x_2 \leq 80$. Unit B binds first, at 80. So the step is 80, and s_B is the variable that leaves. One ratio per rule, take the smallest — that is the whole test. The algebra below just does it for every row at once and handles both bounds.

This is where correctness lives, so it is worth the algebra. Increasing the entering variable by $t \geq 0$ moves the basics:

$$x_B(t) = x_B - t\alpha_q, \quad \alpha_q = B^{-1}A_q$$

Each basic i stays inside $[lo_i, hi_i]$ only while

$$t \leq \begin{cases} \frac{x_{B_i} - lo_{B_i}}{\alpha_{qi}} & \alpha_{qi} > 0 \quad (\text{falling toward its lower bound}) \\ \frac{x_{B_i} - hi_{B_i}}{\alpha_{qi}} & \alpha_{qi} < 0 \quad (\text{rising toward its upper bound}) \end{cases}$$

The smallest such t is the step; the argmin is the leaving variable r . If no α_{qi} has the blocking sign, t is unbounded and so is the LP.

Two practical points that are not decoration:

- **Pivot magnitude matters more than step length.** The chosen α_{qr} becomes a divisor in the basis update; picking a tiny one because it wins the ratio by 10^{-12} poisons the factorisation. The implementation takes the best pivot among candidates within a tolerance of the minimum ratio.
- **Degeneracy.** When x_{B_i} already sits on its bound the ratio is 0, the step is 0, and the objective does not move. Repeated, this cycles. The EXPAND scheme allows a tiny, slowly growing bound relaxation so a strictly positive step is always available.

10.2 Devex, written out

In plain words. Which variable should we bring in? The obvious answer is “whichever earns the most per unit”. But that is like choosing a road by its gradient without asking how long it is: a steep road that ends after ten metres beats nothing, and loses to a gentle road that runs for a kilometre.

What we actually want is gain *per step actually taken*, and the length of the step depends on how strongly that variable disturbs everything else. Measuring that exactly costs as much as taking the step. Devex is the cheap estimate: keep a running guess of each variable’s “disturbance”, and update the guesses from information the pivot already produced.

Dantzig picks $\arg \max_j |d_j|$. That is the steepest *slope*, but the true descent per unit distance moved is $d_j/\|\alpha_j\|$, and computing every α_j is the thing we are trying to avoid.

Devex keeps a reference framework and an approximate weight $w_j \approx \|\alpha_j\|^2$, and prices on

$$\text{score}_j = \frac{d_j^2}{w_j}$$

After a pivot on (r, q) , the weights update from the pivot row alone:

$$w_j \leftarrow \max \left(w_j, \left(\frac{\alpha_{rj}}{\alpha_{rq}} \right)^2 w_q \right), \quad w_q \leftarrow \max \left(\frac{w_q}{\alpha_{rq}^2}, 1 \right)$$

When the weights drift too far from the truth, the framework is reset and all weights return to 1. This is the cheap approximation to steepest edge, and the reason it is affordable is that α_{rj} — the pivot row — is computed once and serves both the weight update and, as Section 10.3 shows, the reduced costs.

10.3 The two refinements that matter most here

Devex pricing. Choosing the entering variable by the most negative d_q (the textbook Dantzig rule) ignores that different columns move the solution by different distances. Devex keeps a running estimate of each column's edge norm and picks by d_j^2/w_j — steepest descent rather than steepest slope.

Incremental pricing. Recomputing every reduced cost each iteration is a full pass over A . But after a pivot the reduced costs update from the pivot row:

$$\theta_d = \frac{d_q}{\alpha_{rq}}, \quad d_j \leftarrow d_j - \theta_d \alpha_{rj}$$

and α_{rj} is exactly what the Devex update already computes. So two passes over the matrix become one. Measured: **0.80× the time** across the Netlib set, with **degen3** going from 85.2 s to 66.2 s.

10.4 The dual simplex — the one that actually runs the MILP

In plain words. There are two ways to reach an optimum, and they are mirror images.

The ordinary (primal) simplex keeps the plan **runnable** at every step and works on making it **profitable**. It walks along the edge of the allowed region, always legal, getting better.

The dual simplex does the opposite: it keeps the plan **profitable-looking** — the prices are already consistent with optimality — and works on making it **legal**. It walks through illegal plans, fixing violations one at a time, and stops when nothing is violated.

Why would you ever want the second one? Because of what happens in branch-and-bound. We solve a problem, then add one new rule ($x_1 \leq 2$) and re-solve. Adding a rule can make the old plan illegal — but it does **not** change any price, because the objective did not change. So the old answer is *already* in exactly the state the dual simplex wants: prices right, one rule broken. It repairs that in a couple of steps instead of solving from scratch.

The primal simplex keeps x feasible and works toward dual feasibility. The **dual simplex** does the mirror: it keeps the reduced costs valid and works toward primal feasibility. Same basis machinery, the two loops swapped:

1. **Choose a leaving variable.** Pick a basic x_{B_r} that violates its bound; the size of the violation (weighted) is the pricing score.
2. **Ratio test on the row.** Compute the pivot row $\alpha_r = e_r^\top B^{-1}A$ and choose the entering column q minimising

$$\frac{d_j}{\alpha_{rj}} \quad \text{over } j \text{ with the correct sign of } \alpha_{rj}$$

The minimum is exactly the largest step that keeps every d_j on the right side of zero.

3. **Pivot.**

Why this is the node solver. After branching adds $x_j \leq 3$, the parent’s optimal basis is still *dual* feasible — the objective did not change, so the reduced costs did not either. Only x_j may now violate a bound. So the child starts one violated bound away from optimal, and the dual simplex repairs it in a handful of pivots.

Measured: **18,772 of 18,775** node relaxations warm start, averaging about **three pivots** each. A cold solve would be hundreds. This single fact is the reason branch-and-bound is affordable, and Section 16.1 turns on it.

And it is where the worst bug lived. Bland’s anti-cycling rule — pick the lowest index among candidates — is safe in the primal, where pricing and the ratio test are separate steps, so overriding pricing changes only *which* good step you take. In the dual they are the same step: the ratio test *is* the entering choice, and it is the only thing keeping the reduced costs signed correctly. Overriding it produced `fit1p` at 33,609 against a true 9,146.38 — feasible, row violation 2.8×10^{-14} , and reported optimal. Removing the override took the suite from 13/16 to **16/16**.

10.5 Underneath everything: the LU factorisation

In plain words. Every iteration needs to solve a system of equations — the same system as last time, with one column swapped. Solving from scratch each time would dominate everything.

The standard trick is to do the elimination *once* and store the recipe. Gaussian elimination on a matrix B produces two triangular tables, L and U , with $B = LU$; and a triangular system is trivial to solve by substituting one unknown at a time. So the expensive part is paid once and every later solve is two cheap sweeps.

Two complications, and both have a name below:

Fill-in. Refinery matrices are almost all zeros — a rule mentions five variables out of ten thousand. Elimination *creates* new non-zeros where there were none, and a bad elimination order can turn a nearly-empty table into a full one. Choosing the order well is worth orders of magnitude, and that is what Markowitz does.

Small pivots. Elimination divides by the entry it eliminates on. Divide by 10^{-14} and the rounding error swamps the answer. So the sparsest choice is sometimes refused for the safer one — that trade is the threshold in the rule.

B^{-1} is never formed. B is factorised as $B = LU$ with a **Markowitz** pivot rule, which at each elimination step picks the entry minimising

$$(r_i - 1)(c_j - 1)$$

among entries large enough to be numerically safe ($|a_{ij}| \geq \tau \max_k |a_{kj}|$), where r_i and c_j are the remaining counts in that row and column. This is a direct trade of sparsity against stability: the product counts how many new nonzeros the elimination could create, and the threshold refuses the sparsest choice when it is too small to divide by.

Between refactorisations, each pivot is absorbed by the **product form**: the new inverse is the old one times an elementary matrix E_k , so

$$B_k^{-1} = E_k E_{k-1} \cdots E_1 U^{-1} L^{-1}$$

and FTRAN/BTRAN simply pass through the accumulated list. The list grows, so the basis is refactorised periodically — every ~100 pivots, or immediately when a stability check fails.

11. The first-order method — the piece that runs on a GPU

11.1 Why a saddle point

In plain words. The simplex walks the *edge* of the region, so it needs to know which rules are tight — bookkeeping that is inherently sequential and that a GPU cannot help with.

The alternative is to stop tracking corners entirely and turn the constraints into a *penalty*. Imagine a referee whose whole job is to punish you for breaking a rule, and who gets to choose how hard. You pick a plan to maximise profit; the referee picks penalties to punish violations. Neither of you can win outright, and the standoff you settle into is exactly the optimal plan with its shadow prices.

That standoff is a **saddle point**: the lowest point along one direction and the highest along another, like the middle of a horse’s saddle or a mountain pass. Writing the problem this way replaces “which rules are tight” with “keep adjusting both sides” — and adjusting is arithmetic, which parallelises.

Write the LP as a min-max problem. For $Kx \geq q$:

$$\min_x \max_{y \geq 0} c^\top x + y^\top (q - Kx)$$

If x violates a constraint, y can push the inner term to $+\infty$, so the outer minimisation will not tolerate it. The saddle point of this is exactly the primal-dual optimum.

11.2 PDHG

In plain words. One iteration is four steps, and each is something you can picture:

1. The planner moves the plan a little in the direction that improves profit, pushing against the current penalties.
2. The plan is **clamped** back inside its own bounds (Section 8) — you cannot produce negative petrol.
3. The referee looks at how badly the rules are now broken and raises the penalties on the broken ones.
4. The penalties are clamped too — a rule that is not tight gets no penalty.

Then repeat, a hundred thousand times. Both sides keep adjusting until neither wants to move, which by Section 11.1 is the answer.

The extrapolated point $\bar{x} = 2x^{k+1} - x^k$ in step 3 is a small but essential trick: the referee reacts to *where the planner is heading* rather than where they were, which is what makes the whole thing converge rather than oscillate.

Primal-dual hybrid gradient alternates a gradient step in each variable, each projected back onto its own constraint:

$$x^{k+1} = \Pi_{[lo,hi]}(x^k - \tau(c - K^\top y^k))$$

$$\bar{x} = 2x^{k+1} - x^k$$

$$y^{k+1} = \Pi_{y \geq 0}(y^k + \sigma(q - K\bar{x}))$$

Look at what is in there: **two sparse matrix-vector products** ($K^\top y$ and $K\bar{x}$), some vector arithmetic, and two clamps. No factorisation, no basis, nothing sequential.

That is the whole reason this method is on a GPU. A matrix-vector product is thousands of independent row dot-products. A clamp is elementwise. Every step is embarrassingly parallel.

The step sizes must satisfy $\tau\sigma\|K\|^2 \leq 1$; the split between them is the **primal weight** ω , with $\tau = \eta/\omega$ and $\sigma = \eta\omega$.

11.3 Restarts and Halpern

In plain words, before the formulas:

- **Restarting** is starting the method again from the good point you have reached. This sounds like it does nothing — same method, same place. It is not nothing, because how fast this method closes in depends on *how far it is from the answer*, and restarting from a closer point resets that distance. It is the difference between a rate that decays and a rate that keeps re-earning itself.
- **Halpern** ties a rubber band from the current point back to where the epoch started. Early on the band pulls hard and stops the iterates wandering; the weight $\frac{1}{k+2}$ shrinks each step, so the band lets go and it becomes ordinary PDHG again.
- **Reflection** is “if that step helped, take some more of it” — go past where the step landed, in the same direction.

Plain PDHG converges slowly. Two accelerations:

Restarting. Periodically take the current iterate (or an average) and begin again from it. Sounds like it does nothing; it changes the convergence rate, because the method’s rate depends on the distance to the optimum and restarting resets that distance.

Halpern iteration. Keep an anchor z^0 from the epoch start and pull each step toward it with a decaying weight:

$$z^{k+1} = \frac{k+1}{k+2}T(z^k) + \frac{1}{k+2}z^0$$

where T is one PDHG step. Early on the anchor pulls hard; later the pull vanishes and it becomes plain PDHG.

Reflection. Overshoot the operator’s output:

$$R(z) = (1 + \gamma)T(z) - \gamma z, \quad \gamma \in [0, 1]$$

This costs nothing extra: $R(z) = T(z) + \gamma(T(z) - z)$, and $T(z) - z$ is already computed.

11.4 What this method actually is

This matters for how the project is described. Chen, Sun, Yuan, Zhang and Zhao, [arXiv:2509.23903](#), **Proposition 3.1**: the reflected restarted Halpern PDHG with $\gamma = 1$ is the Halpern Peaceman–Rachford method, with the semi-proximal term taken as $T_1 = \lambda_A I - AA^*$, $\sigma = \eta/\omega$ and $\lambda_A = 1/\eta^2 \geq \|A\|^2$.

Our default reflection is 1.0. So this solver implements an **HPR method**, not a pile of enhancements to PDHG. That is worth saying precisely, because it is checkable.

The same paper measures the distance to the best implementation, HPR-LP, at accuracy 10^{-8} : on the Mittelman LP benchmark both solve 44 of 49 with HPR-LP about $1.1\times$ faster; on MIPLIB relaxations HPR-LP solves two more and is $1.8\times$ faster. Their conclusion is that both are effective realisations of the same method and **the difference is implementation, not algorithm**.

11.5 Feasibility polishing

First-order methods reach moderate accuracy quickly and high accuracy slowly. For a refinery that is the wrong shape of error: a plan that violates a capacity by 0.8 units cannot be run, even if its objective looks fine.

PDLP’s insight ([arXiv:2501.07018 §4](#)): a **feasibility problem** — the same constraints, no objective — is far easier for this method, because nothing is pulling against the constraints. And PDHG’s iterates are non-increasing in distance to any optimal solution, so a run warm started near a good point stays near it.

So when the duality gap is already acceptable, pause and solve two subproblems warm started from where you are:

primal: $c := 0$, started from $(x, 0)$ **dual:** $q := 0$, finite bounds $:= 0$, started from $(0, y)$

The result is a point that is *almost exactly feasible* with roughly the objective you already had.

The trade is explicit and it is the right one here: it buys feasibility and pays in the duality gap. On the refinery model, at the shipped defaults:

	without polishing	with polishing
iterations	160,720	12,800 + 1,720
capacity violation	1.46e-02	1.28e-04
duality gap	1.1e-09	4.5e-03

Eleven times fewer iterations for a violation 114 times smaller, at the cost of a 0.45% gap. A plan that overruns a unit is not a plan; a plan that leaves half a percent on the table is.

11.6 Scaling — the step that decides whether any of it works

In plain words. Suppose one rule is measured in barrels per day (numbers around 100,000) and the next in sulphur fraction (numbers around 0.01). A step that meaningfully moves the first is invisible

to the second, and a step that moves the second takes forever on the first. But this method uses **one step size for everything**.

The fix is to change units. Divide the barrel row by 100,000 and multiply the sulphur row by 100 and now both are around 1 — same problem, same answer, comfortable numbers. That is all scaling is: choosing units so nothing is enormous and nothing is tiny.

The simplex does not need this nearly as much, because it works with exact equalities and one entering variable at a time. For a first-order method it is load-bearing.

A refinery model mixes barrels (10^5), fractions (10^{-2}) and rupees (10^7) in the same matrix. A first-order method takes a *single* step size for all of it, so a badly scaled matrix means the step is either far too long for one row or far too short for another. Unlike the simplex, this method has no basis to hide behind — scaling is not a nicety here, it is load-bearing.

Two diagonal matrices are found, D_r for rows and D_c for columns, and the problem is replaced by

$$\tilde{A} = D_r A D_c, \quad \tilde{c} = D_c c, \quad \tilde{b} = D_r b$$

with variable bounds divided by D_c . Everything is undone exactly at the end, so scaling changes the path, never the answer.

Ruiz equilibration. Repeat: divide each row by $\sqrt{\|A_{i.}\|_\infty}$ and each column by $\sqrt{\|A_{.j}\|_\infty}$. Each pass drives the largest absolute entry of every row and column toward 1. Ten passes are plenty.

Pock–Chambolle. A second pass tuned for exactly this algorithm, with a parameter α (we use $\alpha = 1$):

$$(D_r)_i = \frac{1}{\sqrt{\sum_j |A_{ij}|^{2-\alpha}}}, \quad (D_c)_j = \frac{1}{\sqrt{\sum_i |A_{ij}|^\alpha}}$$

This is not arbitrary: it is chosen so that the step-size condition $\tau\sigma\|\tilde{A}\|^2 \leq 1$ is satisfiable with a *uniform* step, which is what the algorithm actually needs.

Both are applied, Ruiz first. On the refinery model the matrix norm drops by more than an order of magnitude, and with it the number of iterations.

11.7 When to stop — the termination test

The solver may not stop when “the numbers stop moving”. It stops when it can show the three conditions of optimality are met to a stated tolerance. Unscale first, then measure on the original problem — a small residual in scaled space can be a large one in barrels.

Primal residual — how far the constraints are from being satisfied:

$$r_p = \|\Pi_{[l,u]}(Ax) - Ax\|_2$$

Dual residual — how far the reduced costs are from being consistent with the bounds the variables actually sit at:

$$r_d = \|c - A^\top y - \lambda\|_2$$

where λ is the part of the reduced cost that a variable at a bound is allowed to absorb.

Duality gap — the two objectives must meet (Section 5.4):

$$g = |c^\top x - (q^\top y + \text{bound terms})|$$

Each is tested *relative*, not absolute, because “violation of 0.001” means nothing until you know whether the row’s right-hand side is 1 or 10^6 :

$$r_p \leq \varepsilon (1 + \|b\|), \quad r_d \leq \varepsilon (1 + \|c\|), \quad g \leq \varepsilon (1 + |c^\top x| + |b^\top y|)$$

The gap tolerance is separately settable (`--gap-tol`), and that flag exists for an honest reason: the gap is the slowest of the three to close, and a planner usually wants a strictly runnable plan sooner rather than a provably-last-rupee plan later.

11.8 Proving there is no answer

A solver that only ever says “here is the optimum” is half a tool. If a planner over-constrains a model, the useful reply is “*these constraints cannot all hold*” — proved, not guessed from a stall.

Farkas’ lemma gives the proof object. Exactly one of the following holds:

$$\exists x \geq 0 : Ax = b \quad \text{or} \quad \exists y : A^\top y \leq 0 \quad \text{and} \quad b^\top y > 0$$

The second y is a **certificate**: any non-negative combination of the rows that produces a contradiction. It can be checked in one matrix-vector product by someone who does not trust the solver at all.

The first-order method produces these naturally. When a problem is infeasible its iterates diverge, but the *direction* of divergence converges — the normalised difference $(z^{\wedge\{k+1\}} - z^k)/|z^{\wedge\{k+1\}} - z^k|$ settles onto the certificate. So the solver watches the difference direction alongside the iterate and, when that direction satisfies the Farkas conditions to tolerance, reports infeasibility with the certificate attached. Unboundedness is the mirror image, with the primal difference giving a ray of improvement.

11.9 Step size, primal weight, restarts — the three tuned quantities

In plain words, these three answer:

- **step size** — how big a move to make each iteration. Too small and it crawls; too big and it overshoots and never settles.
- **primal weight** — the plan and the penalties are different kinds of number and do not need equally sized steps. This is the dial that splits one step size between them.
- **restart** — when to stop the current epoch and begin again from here.

Every one of them has a formula that was tuned by measurement, not taken on faith. The sweep table below is the point of the section.

These three are where a first-order LP solver is won or lost, and each is one formula.

Step size. The theory requires $\tau\sigma\|A\|^2 \leq 1$. Two ways to get there:

Adaptive — try a step, measure whether the movement was consistent with the local curvature, and accept or shrink:

$$\bar{\eta} = \frac{\|\Delta z\|_\omega^2}{2|\Delta y^\top A \Delta x|}$$

then accept if $\eta \leq \bar{\eta}$ and propose the next η from $\bar{\eta}$ with a decaying exploration term. Costs one extra product per rejected step.

Constant — estimate $\|A\|_2$ once and set

$$\eta = \frac{\theta}{\|A\|_2}$$

cuPDL Px uses $\theta = 0.998$. Both were implemented and swept over Netlib:

($\|A\|_2$ — the largest factor by which A can stretch any vector — is found by **power iteration**: start from a random vector, multiply by $A^\top A$ over and over, and normalise. Each multiplication amplifies the stretchiest direction most, so the vector lines up with it and the growth factor converges to $\|A\|_2^2$. Twenty multiplications is usually plenty, and each one is the same matrix-vector product the main loop already does.)

θ	instances solved (of 88)
adaptive	76
0.90	75
0.95	74
0.998	74

The literature’s value is not the best value *here*, and adaptive is best of all on this set. The shipped default is adaptive, with the constant path available and $\theta = 0.90$ if it is used. Sweeping was cheaper than believing.

Primal weight. ω splits the single step between primal and dual: $\tau = \eta/\omega$, $\sigma = \eta\omega$. It should track the relative size of movement on the two sides, so the classic update is a smoothed ratio in log space:

$$\omega^{k+1} = \exp\left(\theta_\omega \log \frac{\|\Delta y\|}{\|\Delta x\|} + (1 - \theta_\omega) \log \omega^k\right)$$

cuPDL Px replaces this with a **PID controller** on the log-ratio error $e = \log \|\Delta y\| - \log \|\Delta x\|$:

$$\log \omega^{k+1} = \log \omega^k + K_p e + K_i \sum e + K_d (e - e_{\text{prev}})$$

Swept here: $K_p = 0.5$, $K_i = 0$, $K_d = 0.3$. The integral term was measured and set to zero — it accumulates and overshoots on this set.

Restarts. cuPDLP’s rule restarts the epoch when any of three conditions fires, on a normalised duality gap μ measured over the epoch:

$$\begin{aligned} \text{sufficient decay: } & \mu_c \leq 0.2 \mu_0 \\ \text{necessary decay, and no further progress: } & \mu_c \leq 0.8 \mu_0 \text{ and } \mu_c > \mu_c^{\text{prev}} \\ \text{epoch already long: } & k \geq 0.36 K \end{aligned}$$

cuPDLPx simplifies this to a **fixed-point residual** test — restart when $\|z - T(z)\|$ has fallen by a fixed factor since the epoch began. Cheaper, since $z - T(z)$ is already computed for the reflection, and it does not need the gap. Both are implemented; the fixed-point rule is the default.

12. Branch and cut, concretely

Section 7.2 gave the skeleton: relax, branch, bound, prune. This section is the four things that make the bound tighter — because by Section 7.2 the bound is the only thing that matters.

12.1 Cuts

In plain words. A cut is a rule that was *always* true for whole-number plans, but which the LP relaxation did not know. Adding it does not remove a single integer solution — but it does chop off the fractional point the LP handed back, forcing a better bound next time.

Cover cut, on numbers. A bag holds 10 kg. Three items weigh 6, 7 and 8 kg, and $x_j = 1$ means “take item j ”:

$$6x_1 + 7x_2 + 8x_3 \leq 10, \quad x_j \in \{0, 1\}$$

Any *two* of these exceed 10 ($6 + 7 = 13$, $6 + 8 = 14$, $7 + 8 = 15$). So at most one item can be taken:

$$x_1 + x_2 + x_3 \leq 1$$

That is a **cover cut**. It is obvious to a person and invisible to the LP, which is perfectly happy with $x = (0.6, 0.4, 0.4)$: the weight is $3.6 + 2.8 + 3.2 = 9.6 \leq 10$, so the original row is satisfied. But the cut says $0.6 + 0.4 + 0.4 = 1.4 \leq 1$, which is false — so that point is gone, and the LP is forced to return something closer to an actual whole-number plan.

Rounding cut, on numbers. Take a row that the LP produced:

$$x + s = 2.5, \quad x \text{ whole, } s \geq 0$$

Since $s \geq 0$ we get $x \leq 2.5$, and since x is whole, $x \leq 2$. Feed that back: $s = 2.5 - x \geq 0.5$. **The leftover must be at least 0.5** — a fact the original row never stated, and one that cuts off the LP’s answer of $s = 0$.

That single move — use integrality to round, then read the consequence back out — is the whole idea behind Gomory and MIR cuts. Everything below is that move done carefully enough to survive continuous variables and negative coefficients.

A **cutting plane** is an inequality valid for every integer feasible point but violated by the current fractional relaxation solution. Adding it tightens the bound without removing any answer.

Cover cuts. For a knapsack row $\sum_j a_j x_j \leq b$ with binary x_j and $a_j > 0$, a set C is a *cover* if $\sum_{j \in C} a_j > b$ — the items cannot all be taken. Therefore

$$\sum_{j \in C} x_j \leq |C| - 1$$

MIR (mixed integer rounding). For $\sum_j a_j y_j \leq b$ with $y \geq 0$ and y_j integer for $j \in I$, let $f = b - \lfloor b \rfloor$ and $f_j = a_j - \lfloor a_j \rfloor$. Then

$$\sum_{j \in I} \left(\lfloor a_j \rfloor + \frac{\max(0, f_j - f)}{1 - f} \right) y_j + \sum_{j \notin I} \frac{\min(0, a_j)}{1 - f} y_j \leq \lfloor b \rfloor$$

Gomory mixed-integer cuts apply the same rounding to a row of the simplex tableau rather than a row of the model, which lets them see combinations the model's own rows cannot.

12.2 Reduced-cost fixing

In plain words. Section 5.6 gave us a per-unit price for moving any variable away from where it sits. Suppose the best plan found so far earns 100, and this branch's LP bound says the best it could possibly do is 108 — a slack of 8. If moving x_j costs 4 per unit, then x_j can move at most 2 units before this branch is definitely worse than what we already have. So tighten its bound to $l_j + 2$ and hand that to the children. Free information, no LP solved.

Once there is an incumbent with objective z_{inc} and a node with bound z_{node} , a nonbasic variable at its lower bound with reduced cost $d_j > 0$ costs at least d_j per unit moved. So moving it further than

$$x_j \leq l_j + \frac{z_{\text{inc}} - z_{\text{node}}}{d_j}$$

cannot beat the incumbent. Tighten the bound for the children.

Measured: **gt2** from 7,901 nodes to **1,167**.

12.3 Node propagation

In plain words. Branching pinned $x_1 \leq 2$. Now walk each rule and ask what that implies. In our refinery, if $x_1 \leq 2$ then Unit A's rule $2x_1 + x_2 \leq 100$ forces $x_2 \leq 96$, and if some other rule needed $x_2 \geq 97$ the branch is dead — proved, without solving anything. It is the same interval arithmetic a person does in their head, run over every rule a few times until nothing more tightens.

After branching pins $x_j \leq 3$, interval arithmetic on each row can often tighten other variables — and sometimes prove the child infeasible before its LP is ever solved. For a row $\sum_j a_j x_j \geq q$, the

maximum activity is

$$\text{maxact} = \sum_{a_j > 0} a_j hi_j + \sum_{a_j < 0} a_j lo_j$$

and if $\text{maxact} < q$ the row cannot be satisfied.

Measured: **flugpl** from 28,917 nodes to **477**.

12.4 Reliability branching

Pseudocost branching estimates, from history, how much the bound rises when a variable is branched. But a variable never branched has no history and gets the same optimistic guess as every other — so decisions near the root, which shape the whole tree, are made blind.

Reliability branching strong-branches (actually solves both children, with a small iteration budget) until a variable's pseudocost is trustworthy, then trusts it.

The measured subtlety: it must be **capped by depth**. Unlimited, it made **gt2** nearly three times *worse* (783 $\rightarrow$ 2,225 nodes). Capped at depth 10 it is 194. Near the root a decision shapes the tree; deep down it settles a subtree about to be pruned anyway, and the greedy one-level-ahead choice is not the one that makes the smallest tree.

13. QP by ADMM, written out

13.1 The splitting

The problem is

$$\min_x \frac{1}{2} x^\top Q x + c^\top x \quad \text{s.t.} \quad l \leq A x \leq u$$

The difficulty is that x appears in both a smooth objective and a hard constraint. **Splitting** separates them: introduce a copy $z = A x$ and put the constraint on the copy.

$$\min_{x,z} \frac{1}{2} x^\top Q x + c^\top x + \mathcal{J}_{[l,u]}(z) \quad \text{s.t.} \quad A x = z$$

where $\mathcal{J}$ is zero inside the box and $+\infty$ outside. Now each half is easy on its own: the x half is a linear solve, the z half is a clamp. Alternating direction method of multipliers alternates them, with a dual variable y enforcing the link:

$$\begin{aligned} (\tilde{x}^{k+1}, \tilde{z}^{k+1}) &\leftarrow \text{solve the linear system below} \\ x^{k+1} &= \alpha \tilde{x}^{k+1} + (1 - \alpha) x^k \\ z^{k+1} &= \Pi_{[l,u]}(\alpha \tilde{z}^{k+1} + (1 - \alpha) z^k + \rho^{-1} y^k) \\ y^{k+1} &= y^k + \rho(\alpha \tilde{z}^{k+1} + (1 - \alpha) z^k - z^{k+1}) \end{aligned}$$

α is relaxation (1.6 works well), ρ the penalty on the link.

13.2 The linear system, and why it always factorises

In plain words. ADMM's cost is one system of equations per iteration, and the same system every time. So the question is whether we can eliminate it once and reuse the recipe forever.

Normally you cannot promise that. Gaussian elimination can hit a zero on the diagonal and be forced to swap rows, and the swap depends on the *numbers*, which means you cannot decide the ordering in advance. That kills the reuse.

Here we get a promise. The matrix has a special shape — positive on the top block, negative on the bottom — and for that shape there is a theorem saying **elimination never hits a zero, in any order**. So the order can be chosen purely to keep the matrix sparse, computed once before any arithmetic, and reused for every iteration. That promise is the reason this method is fast.

Every iteration solves the same KKT system:

$$\begin{bmatrix} Q + \sigma I & A^\top \\ A & -\rho^{-1}I \end{bmatrix} \begin{bmatrix} \tilde{x} \\ \nu \end{bmatrix} = \begin{bmatrix} \sigma x^k - c \\ z^k - \rho^{-1}y^k \end{bmatrix}$$

Note what changes between iterations: **only the right-hand side**. The matrix is fixed as long as ρ is. So it is factorised once and every subsequent iteration is two triangular solves.

The matrix is **quasi-definite**: the $(1, 1)$ block $Q + \sigma I$ is positive definite (because $\sigma > 0$, even if Q is only positive *semi*-definite) and the $(2, 2)$ block $-\rho^{-1}I$ is negative definite.

Vanderbei's theorem: every symmetric permutation of a quasi-definite matrix has an $LDL^\top$ factorisation. That is a strong statement — it means no pivot will ever be zero, no matter what ordering is chosen. So the ordering can be picked *purely* to minimise fill (AMD on the sparsity pattern), computed once, and reused. No numerical pivoting, no symbolic re-analysis per iteration.

That is the entire reason this method is fast, and it is why σ exists at all — it is not regularisation for its own sake, it is what buys the theorem.

13.3 The factorisation itself

In plain words. The factorisation is done in two passes, and the split is the clever part.

The **first pass touches no numbers at all** — it only looks at *where* the non-zeros are and works out where the non-zeros of the answer will be. It does this by building a small tree (“if I eliminate row 7, which row does that disturb next?”), and walking that tree gives the answer's shape directly.

The **second pass** then fills those known slots with actual numbers. Because the shape was settled first, memory is allocated exactly once and there is no searching during the arithmetic. It also means the first pass runs *once* while the second may run again if ρ changes.

Up-looking sparse $LDL^\top$ (Davis, Algorithm 849). Two phases:

Symbolic. Build the **elimination tree**: $\text{parent}(k)$ is the row of the first off-diagonal nonzero in column k of L . Walking from a nonzero of K up this tree to the already-computed part gives exactly the nonzero pattern of that row of L — without any floating point. This yields the column counts, so the numeric phase allocates once.

Numeric. For each row k , solve a sparse triangular system against the rows already computed, then

$$D_k = K_{kk} - \sum_{j \in \text{pattern}} L_{kj}^2 D_j$$

Measured against solving the same system iteratively with conjugate gradients: **1.52× faster**, and without CG's dependence on conditioning.

13.4 Adaptive ρ , and where it breaks

ρ balances how hard the link is enforced. Too small and the copy drifts from Ax ; too large and the objective is ignored. The standard rule rescales it from the ratio of the two residuals:

$$\rho \leftarrow \rho \sqrt{\frac{\hat{r}_{\text{prim}}}{\hat{r}_{\text{dual}}}}, \quad \hat{r}_{\text{prim}} = \frac{\|r_{\text{prim}}\|}{\max(\|Ax\|, \|z\|)}, \quad \hat{r}_{\text{dual}} = \frac{\|r_{\text{dual}}\|}{\max(\|Qx\|, \|A^\top y\|, \|c\|)}$$

and refactorises when it changes (hence the gate: only rescale when the ratio exceeds 5, so the factorisation is not thrown away every iteration).

This is the part that fails on five instances, four of them the PRIMALC family. Traced: ρ falls from 10^{-1} to 1.9×10^{-5} within 500 iterations, and the same gate that stops it oscillating also stops it climbing back. Section 16.8 records the two fixes that followed from that diagnosis and the measurements that killed both.

13.5 Polishing

ADMM converges to moderate accuracy quickly, like every first-order method. But once it has converged, the *active set* — which constraints are tight — is usually exactly right even when the numbers are not. So: guess the active set from the converged y , form the reduced KKT system with only those rows as equalities, and solve it directly. If the result satisfies the original constraints, keep it. It is the same trade as feasibility polishing in Section 8.5 — use the cheap method to find the *structure*, then solve the small exact problem that structure implies.

14. Presolve, with the algebra

In plain words. Before solving anything, read the model and delete the parts that were never in question.

A rule like $3x_1 \leq 12$ mentions one variable — that is not a rule, that is a bound, $x_1 \leq 4$. Delete the row. A variable that appears in no rule at all is decided purely by its own price — set it and delete the column. A rule whose worst case still satisfies it can never bind — delete it. Do this repeatedly, because each deletion can expose the next.

On the standard Netlib test set this removes about **19% of rows and 12% of columns** without changing a single answer. It is the cheapest speedup in the solver — and the most dangerous, because every reduction is a claim that certain solutions can be discarded, and one wrong claim discards the answer. The very first wrong answer this project produced came from here.

Eleven reductions are implemented. Here is what each actually computes.

Empty row. No entries. Either the bounds contain zero (drop it) or they do not (the model is infeasible, and this is a proof).

Singleton row. One entry: $ax_j \in [l, u]$ is just a bound on x_j . Intersect it with the existing bound:

$$lo_j \leftarrow \max\left(lo_j, \frac{l}{a}\right), \quad hi_j \leftarrow \min\left(hi_j, \frac{u}{a}\right)$$

with the two swapped when $a < 0$. Then delete the row. Postsolve must restore the row's dual, which is $y = d_j/a$ where d_j is the reduced cost the reduced problem reports for that column.

Empty column. x_j appears in no row, so only the objective decides it: fix it at whichever bound c_j prefers. Unbounded in that direction means the LP is unbounded.

Fixed column. $lo_j = hi_j$. Substitute the value into every row's bounds and delete the column:

$$l_i \leftarrow l_i - A_{ij} lo_j, \quad u_i \leftarrow u_i - A_{ij} lo_j$$

Forcing row. Compute the row's activity range from the variable bounds:

$$\text{minact} = \sum_{a_{ij}>0} a_{ij} lo_j + \sum_{a_{ij}<0} a_{ij} hi_j, \quad \text{maxact} = \sum_{a_{ij}>0} a_{ij} hi_j + \sum_{a_{ij}<0} a_{ij} lo_j$$

If $\text{minact} = u_i$, then *every* variable in the row must be at the bound that achieved the minimum — there is no slack anywhere. Fix them all and drop the row. If $\text{minact} > u_i$ or $\text{maxact} < l_i$, the model is infeasible. And if the range lies strictly inside $[l_i, u_i]$, the row can never bind and is **redundant** — delete it.

This is the reduction that caused the first wrong answer. My own 10^{-9} padding on the bounds manufactured a forcing row where none existed. Fixed by separating two tolerances: a tight one to *fire* a reduction, a looser one to *declare infeasibility*, and declining to act in the band between them.

Doubleton equation. A row $ax_i + bx_j = c$ gives $x_i = (c - bx_j)/a$. Substituting into every other row k containing x_i :

$$A_{ki}x_i = \frac{A_{ki}c}{a} - \frac{A_{ki}b}{a}x_j$$

so row k 's bounds shift by $-A_{ki}c/a$ and x_j 's coefficient there changes by $-A_{ki}b/a$. The objective picks up c_i the same way.

Only applied when x_i is **implied free** — the row plus x_j 's bounds already confine x_i strictly inside its own bounds — because then x_i can never sit *at* a bound, so its reduced cost is zero, and the eliminated row's dual comes back exactly:

$$y_r = \frac{c_i - \sum_{k \neq r} A_{ki} y_k}{a}$$

Without implied-freeness the dual is not recoverable and the shadow prices of Section 5.5 would be quietly wrong.

Two bugs lived here. The staleness guard ran *after* the liveness check — but a column that has been substituted away is also a dead column, so the guard never fired. And a column that survived the substitution could be eliminated by a later reduction, leaving `stocfor2` with 189 units of activity sitting outside the model.

Dual fixing. Count each column’s **locks**: how many rows could be violated by moving x_j up, and how many by moving it down. If nothing can be violated by moving it *down* and $c_j \geq 0$, then some optimal solution has x_j at its lower bound — pushing it down is free and never worse. Fix it there. The mirror case fixes at the upper bound.

This generalises the empty-column rule: that one fires when *nothing at all* can stop the column; this one when nothing can stop it *in the direction the objective already prefers*.

Free/implied-free column substitution, duplicate rows, and singleton-column-in-equality round out the eleven.

14.1 Postsolve is a replay, not an inverse

Reductions are pushed onto a stack as they fire. Postsolve pops them in reverse, each one reconstructing what it removed. Primal recovery is mechanical.

Dual recovery is not. Some reductions invert exactly (singleton row, doubleton with implied-freeness); some do not. The implementation carries a `dual_is_exact()` flag per reduction, and when a run contains an inexact one the solver *says so* rather than reporting a shadow price it cannot stand behind. Given Section 5.5 — a planner may be sizing capital on these numbers — that is not pedantry.

15. How the pieces fit together

For an **LP**: read $\rightarrow$ presolve $\rightarrow$ scale $\rightarrow$ solve (simplex, or first-order on the GPU) $\rightarrow$ unscale $\rightarrow$ postsolve $\rightarrow$ verify optimality conditions $\rightarrow$ report x , y , and the reduced costs.

For a **MILP**: presolve once at the root, solve the root relaxation, then repeatedly — select a node, propagate bounds into it, solve its relaxation with the **dual simplex warm started from the parent** (Section 10.4), separate cuts if it is worth it, apply reduced-cost fixing against the incumbent, and either prune it or branch. The LP solver is called tens of thousands of times; everything in Section 12 exists to reduce that count.

For a **QP**: presolve $\rightarrow$ scale $\rightarrow$ ADMM with a single cached $LDL^\top \rightarrow$ polish.

The GPU sits underneath as a backend: the same first-order algorithm, with the vectors resident on the device and the matrix-vector products, reductions and clamps as kernels. Measured **2.70 $\times$ –7.09 $\times$** over the same algorithm on CPU on a Tesla T4. Two things learned the hard way — `cudaMalloc` does *not* zero its memory where the CPU allocator value-initialises (Section 17), and on one instance **79% of solve time was outside the kernels**, in the convergence check that still runs on the host. That is the top remaining performance item.

Part IV — Every option that was rejected, and why

This is the part that is usually missing from a project write-up. Each of these is standard, respectable and in the textbooks. Each was measured against *this* instance set and each lost.

16. Rejected, with the numbers

16.1 Interior point methods — the full argument

The plan disposed of this in eight words. Here it is from every side.

Against, the obvious way. An IPM needs a sparse Cholesky of $ADA^\top$ (or an $LDL^\top$ of the augmented system) at *every* iteration, with a fill-reducing ordering. Large machinery that nothing else reuses.

Against, the stronger way — it serves neither goal:

- **MILP needs warm starts.** The dual simplex restarts from the parent’s basis and finishes a child in about **three pivots** — 18,772 of 18,775 relaxations warm start here. An IPM does not warm start usefully, so every node solves from scratch. Branch-and-bound as built would not survive that.
- **The GPU needs matrix-vector products.** An IPM’s time is in the sparse factorisation, which is the part that parallelises worst.

For — and this is real. First-order methods converge *linearly*, which is exactly where this solver is weakest. It is why `--gap-tol` exists and why feasibility polishing had to be built. A second-order method gives high accuracy natively. The recent literature makes this argument explicitly.

And the ground has moved:

- NVIDIA shipped **cuDSS**, a GPU direct sparse solver with Cholesky, $LDL^\top$ and LU — the factorisation an IPM needs is now on the device.
- **Condensed-space IPM** reshapes the KKT system into symmetric positive definite form, which factorises far better on a GPU ([arXiv:2405.14236](https://arxiv.org/abs/2405.14236)).

Does the decision still hold? Yes — for a different reason than originally given. Not because “IPM cannot work on a GPU” (less true every year), but because:

1. The accuracy an IPM would add is **already covered by the simplex**, which gets 16 of 16 Netlib instances with published optima.
2. Even with cuDSS, reported gains for fully GPU-based interior-point LP solvers **remain modest** — the sparse factorisation is still the bottleneck.
3. It still would not warm start for branch-and-bound.

Where the honest concession is: QP. For LP, an IPM overlaps methods we already have. For MILP, it is actively worse. But commercial solvers use barrier for QP, and our five QP failures are step-size tuning failures that an IPM would simply not have — it takes Newton steps and has no step-size parameter to mistune. “*IPM would have been better for QP*” is a true statement. It was not built because QP is the smallest of the three components and the machinery is three to four weeks.

What would change it: the refinery model growing to where simplex becomes impractical *and* tight accuracy is required. Written down so the decision gets revisited on evidence.

16.2 Hypersparsity

Hall and McKinnon report a **5.2× mean speedup** from exploiting it. Their criterion: an instance is hyper-sparse when more than 60% of FTRAN/BTRAN results have density under 10%.

Measured here over 21 Netlib instances:

	ftran <10%	btran <10%	combined
czprob	99.6%	43.2%	75.6%
80bau3b	80.2%	65.0%	72.9%
pilot87	3.4%	4.7%	4.1%
degen3	6.1%	10.5%	8.5%
all 21	13.1%	17.5%	15.4%

Four of twenty-one clear the threshold. Their 5.2× came from a test set *selected* for the property (KEN-18, PDS-20, STOCFOR3 — network-structured). Netlib is not that. Six to eight weeks of work to help four instances.

16.3 Forrest–Tomlin updates

A sampling profile of **degen3**, the slowest instance at 66 s, puts 67% of samples in one place and `LuFactor::factorize` at **11 samples out of ~4,500**. Basis updates are not where the time goes. Aimed at something that is not hot.

16.4 Bound-flipping ratio test

Needs boxed columns. Measured: **pilot87** 36.8%, **80bau3b** 35.6%, **fit1p** 23.8% — and **0%** for **afiro**, **sctap1**, **degen3**, **25fv47**, **woodw**, **stocfor2** and **maros-r7**.

16.5 Coefficient tightening

Implemented in full. Fires **three times** across seven MIPLIB instances, all on one, and moves that instance’s root bound from 6875.00000004 to 6875.00000002 — which is noise. Kept in the source, switched off, with the measurement beside it.

16.6 Parallel column merging

20% of Netlib columns are parallel to another — 31,954 of 159,369, all continuous, which is the safe case. Compelling until the merge condition is applied: it also needs the objective in the same ratio, which cuts it to **1,471**. **standata** goes from 606 parallel to 12 mergeable.

0.9% of columns, against needing a new postsolve entry kind that writes two columns from one merged value — the most dangerous part of this codebase to extend. Measured, not built.

16.7 Harris ratio test without EXPAND

Three instances better, six worse, and **blend** stopped solving entirely.

16.8 Two fixes for a QP convergence failure

Five QP instances fail, four of them the PRIMALC family whose DUALC counterparts all solve. Diagnosis was clear: the adaptive step-size rule drives ρ from 10^{-1} to 1.9×10^{-5} within 500 iterations and the gate that stops it oscillating also stops it recovering.

Two fixes follow from that diagnosis. **Neither worked.** Limiting the drift made things monotonically worse (35/40 unlimited, 33 at a factor of 100, 30 at 10). Relaxing the gate after a long stall changed nothing at all. Both reverted, with the measurements kept where the option would have been.

Part V — What went wrong, and what it taught

17. Six times the solver was confidently wrong

The plan's risk register had eleven risks and ten were about *finishing on time*. That risk never materialised. This one did, repeatedly.

Presolve declared a feasible model infeasible. My own 1e-9 bound padding manufactured forcing rows. Fixed by using two tolerances — one to fire a reduction, a looser one to declare infeasibility — and declining to reduce in between.

The dual simplex reported a wrong answer as optimal. `fit1p` at 33,609 against a true 9,146.38, feasible, row violation 2.8e-14. Cause: Bland's anti-cycling rule was overriding the dual's ratio test. In the primal, pricing and the ratio test are separate steps so overriding pricing is safe; in the dual the ratio test *is* the entering choice and the only thing keeping reduced costs on the right side of zero. Removing the override: 13/16 → **16/16**.

A cover cut removed feasible points. The separator inferred whether an item was complemented by testing `slack == x_j` — true for complemented items, false otherwise, *except* at $x_j = 0.5$ where $1 - x_j$ is also 0.5 and everything reads as complemented. A simplex vertex can sit a binary at exactly 0.5. 426 separations at random interior points never landed there.

An $LDL^\top$ silently became a diagonal factorisation. Handed the upper triangle where the algorithm needs the lower. Every entry skipped, elimination tree empty. The diagonal test passed at 1e-14 and said nothing.

The GPU never uploaded the dual iterate. Harmless while everything started at zero — and silently fatal for any warm start, because on CUDA the loop began from whatever `cudaMalloc` returned. The CPU allocator zeroes; CUDA does not.

fiber returned a proved optimum 60.8% wrong. 652,748.78 against a true 405,935.18, with a matching dual bound and no complaint. Two separate bugs:

1. A bound crossing of 1.78×10^{-15} — the last bit of a double — treated as proof that a box was empty. Bound propagation tolerates crossings up to 10^{-7} ; the infeasibility check tolerated nothing. **The two disagreed about what an empty box is** and the answer fell through the gap.
2. A cover cut derived from an *earlier cut* rather than a model row. Each cut family was valid alone — only the combination broke, because only the combined run reached the point that produced the bad cut.

18. The tools that exist because something got through

- **A dual-feasibility check** the simplex runs on itself before claiming optimality. It found the Bland bug on its first run.
- **Cut validity by enumeration** over small programs, separating at simplex vertices as well as random points.
- **An $LDL^\top$ test** that builds K , picks x , forms $b = Kx$ and compares — never a residual the factorisation computed about itself.
- **A survey against 103 published optima** instead of seven. It found the `fiber` wrong answer within minutes of existing.

- **A debug-solution tracker.** Hand the solver an answer known to be correct; every point that can discard a node first checks whether that answer is inside it, and names the first prune that throws it away. SCIP carries the same facility. It found both **fiber** bugs in minutes after four layers of manual elimination had found neither.

19. Traps in measuring, not in the code

- **zsh does not word-split unquoted variables.** A variable holding two flags arrives as one argument, the program rejects it, and the checker silently reads a stale file. Cost three separate debugging sessions.
- **A measurement under load is not a measurement.** Twice a benchmark reported a regression that did not exist. Both times the change was a pure improvement.
- **A binary linked against a static library does not relink when the library is rebuilt.** Twenty minutes reading correct code that was not being executed.

Part VI — Where it stands and what is next

20. Verified against published answers

- Reader matches HiGHS on all 88 Netlib instances, $1.4\text{--}1.5\times$ faster.
- Primal and dual simplex both correct on all 16 Netlib instances with published optima.
- Presolve removes $\sim 19\%$ of rows and $\sim 12\%$ of columns without changing any answer, and recovers duals as well as primals.
- GPU measured $2.70\times\text{--}7.09\times$ over the same algorithm on CPU (Tesla T4).
- Branch-and-cut proves optimality on the smaller MIPLIB instances; on several it does not prove, the *solution* is already the published optimum.
- QP solves 35 of the 40 smallest Maros–Mészáros instances.

21. The benchmark to be judged against

Mittelman’s [LPfeas benchmark](#) is where cuPDLPx and HPR-LP — the two papers this method comes from — are actually scored. Over 65 problems, cuPDLPx solves 57, HiGHS 55, and OR-Tools’ PDLP 50.

That last number is worth sitting with. **OR-Tools’ PDLP is the same algorithm family implemented here, and it solves the fewest of any code except KNITRO.** The method is not automatically good. The implementation is most of it.

Eight of its forty public instances are already here. Reporting solved-or-not at 10^{-6} against that published table is a comparison a reader can check.

22. What is next, in order

1. **Verify on a GPU.** Nothing built since the last run has been tested on hardware, and the CPU hides a class of bug that only appears on the device. This closes a risk; it is not a feature.
2. **Re-fit the branch-and-bound constants against 103 instances, not seven.** Every one of them was chosen against seven, and the wider set found a wrong answer within minutes.
3. **Move the convergence check onto the device.** On one instance 79% of solve time is outside the kernels. Half done.
4. **Report against the LPfeas table.**
5. **Continuous integration.** The test suite is good; nothing runs it automatically.

Deliberately **not** on that list, each for a measured reason in Part IV: interior point, hypersparsity, Forrest–Tomlin, bound-flipping ratio test, parallel columns, coefficient tightening.

23. The one thing worth taking away

The algorithms were not the hard part. They are in papers and they work.

What took the time was finding out **when the solver was quietly wrong** — and building the things that make that visible.

A slow solver tells you it is slow. A wrong one tells you nothing: it hands back a confident number with a matching bound and no complaint. On **fiber** it was wrong by 60% and looked completely healthy.

Every checking tool in Section 18 exists because something got through.